

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing COURSE CODE: CSA37

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Student Name	V. Venkata Kumar Reddy
Register Number	192311350
Course	Software Testing (CSA3712)
Assessment	CO1 - AT1, Weightage 50%
Questions Answered	All 20 questions with answers

Code of Conduct:

*I **V. Venkata Kumar Reddy / 192311350** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V. Venkata Kumar Reddy

Register No: 192311350

PART A - QUESTIONS

Question 1 (Bloom's Level 3)

A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Question 2 (Bloom's Level 3)

An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Question 3 (Bloom's Level 3)

A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Question 4 (Bloom's Level 3)

A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Question 5 (Bloom's Level 3)

A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Question 6 (Bloom's Level 3)

A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Question 7 (Bloom's Level 3)

A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Question 8 (Bloom's Level 3)

An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Question 9 (Bloom's Level 3)

A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Question 10 (Bloom's Level 3)

A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Question 11 (Bloom's Level 3)

A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Question 12 (Bloom's Level 3)

A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Question 13 (Bloom's Level 3)

A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Question 14 (Bloom's Level 3)

A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Question 15 (Bloom's Level 3)

A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Question 16 (Bloom's Level 3)

A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Question 17 (Bloom's Level 3)

A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Question 18 (Bloom's Level 3)

A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Question 19 (Bloom's Level 3)

A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Question 20 (Bloom's Level 3)

A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

PART B - ANSWERS

Answer 1

1. Introduction

Testing is not a last-minute activity of "trying the application and seeing what breaks". It is an engineering activity: it is planned, designed, measured, executed with tools and improved continuously. In a banking application, where every transaction involves money and regulatory compliance, ad-hoc testing leads to missed defects, duplicated effort and failures in production. The redesign therefore replaces random checking with a disciplined, repeatable test process.

2. Redesigned Testing Approach

- **Test planning:** Define scope, objectives, risks, entry/exit criteria, schedule, resources and the test environment before any execution begins (as in the IEEE 829 test plan).
- **Systematic test design:** Derive test cases from requirements using black-box techniques (equivalence partitioning, boundary value analysis, decision tables, state transition) and supplement them with white-box coverage (statement, branch, path).
- **Traceability:** Map every requirement to one or more test cases through a Requirement Traceability Matrix (RTM) so that no feature is left untested.
- **Test execution and logging:** Run cases in a controlled environment, record actual results, and log every mismatch as a defect with severity and priority.
- **Review and improvement:** Hold test-case reviews and defect-causal analysis at the end of each cycle to improve the next one.

3. Introducing Tools and Standards

- **Tools:** Test management (TestRail, Jira/Zephyr), defect tracking (Jira, Bugzilla), automation (Selenium, Appium, JUnit/TestNG), API testing (Postman, REST Assured), performance testing (JMeter) and CI integration (Jenkins).
- **Standards:** IEEE 829 for test documentation, ISO/IEC/IEEE 29119 for the testing process, ISTQB terminology, and organisational templates for test cases and defect reports.
- **Environment and data:** A dedicated test environment with masked production-like data so that real customer data is never exposed (PCI-DSS / data-privacy compliance).

4. How the Approach Improves Defect Detection and Quality

Because test cases are derived from requirements and techniques, coverage becomes measurable and gaps become visible. Defects are found earlier (cheaper to fix), the same defect is not missed in repeated cycles because regression suites are automated, and metrics such as defect detection

percentage, test coverage and defect density give management real quality data instead of opinions.

Aspect	Ad-hoc Testing	Structured Testing
Planning	None or informal	Documented test plan with criteria
Test cases	Created on the fly	Designed using techniques, reviewed
Coverage	Unknown	Measured through RTM and code coverage
Repeatability	Poor	High, supported by automation
Defect detection	Random, low	Systematic, high

5. Techniques to Replace Ad-hoc Testing

Equivalence partitioning and boundary value analysis for amount and account fields; decision tables for business rules such as overdraft and daily limits; state transition testing for account status (active, blocked, dormant); use-case/scenario testing for end-to-end journeys; error guessing based on past defects; and risk-based testing to give more effort to high-risk modules such as payments.

6. Practical Example: Fund Transfer

Requirement: a customer can transfer Rs. 1 to Rs. 50,000 per transaction. Using boundary value analysis, the test cases are 0, 1, 2, 49,999, 50,000 and 50,001. Using equivalence partitioning, the classes are invalid (below 1), valid (1 to 50,000) and invalid (above 50,000). Decision-table tests combine balance sufficient/insufficient with limit within/exceeded. An ad-hoc tester typically tries only "Rs. 5,000" and misses the defect at 50,000 where the system wrongly rejects the transfer. The structured design catches it before release.

7. Conclusion

Treating testing as an engineering activity, with planning, technique-based design, tools and standards, converts unpredictable testing into a measurable and repeatable process. The banking team will see fewer production failures, faster defect detection and higher customer trust.

Summary of Key Points

- Testing must be planned, designed, executed and measured like any other engineering activity.
- Tools (test management, automation, defect tracking) and standards (IEEE 829, ISO/IEC/IEEE 29119) give consistency and traceability.
- Technique-based design (BVA, equivalence partitioning, decision tables) finds defects that ad-hoc testing misses.

- Outcome: fewer production failures, measurable quality and lower cost of defects in the banking application.

Answer 2

1. Introduction

A defined testing process gives every release the same set of activities, entry and exit criteria, and deliverables. Without it, one release may be tested thoroughly and another barely at all, which explains the inconsistent ATM results. The solution is to align testing activities with each phase of the Software Development Life Cycle (SDLC), following the V-model idea that every development phase has a matching test activity.

2. Structured Testing Process Across SDLC Phases

SDLC Phase	Testing Activity	Relevance to Quality
Requirements	Requirement review, testability check, acceptance criteria	Removes ambiguous or missing requirements early
Design	Design review, test planning, test-case design	Avoids design flaws; tests ready before code
Coding	Unit testing, code reviews, static analysis	Finds logic errors at lowest cost
Integration	Integration testing of card reader, bank server, cash dispenser	Detects interface and data-flow defects
System	Functional, performance, security, recovery testing	Validates the complete ATM against requirements
Acceptance	User acceptance testing, field trials	Confirms business needs are met
Maintenance	Regression and confirmation testing	Ensures fixes do not break working features

3. Stages of the Test Process

- **Planning and control:** Define objectives, scope, effort and exit criteria; monitor progress against the plan.
- **Analysis and design:** Convert requirements into test conditions and detailed test cases with expected results.
- **Implementation:** Prepare test data (cards, PINs, account balances), the simulator and the environment.
- **Execution:** Run cases, compare actual and expected results, and log defects.
- **Evaluation and reporting:** Check exit criteria, summarise coverage and defects, and decide on release.
- **Closure:** Archive test ware, record lessons learned and update the process.

4. Application to ATM Withdrawal

The withdrawal function involves card insertion, PIN validation, amount entry, balance check, cash dispensing, receipt and account update. Sample test cases are given below.

Test Case	Input	Expected Result
TC1 Valid withdrawal	Valid card, correct PIN, Rs. 2,000, balance Rs. 10,000	Cash dispensed; balance Rs. 8,000; receipt printed
TC2 Insufficient funds	Rs. 12,000 with balance Rs. 10,000	Message shown; no cash dispensed; balance unchanged
TC3 Wrong PIN x3	Three incorrect PIN entries	Card retained or blocked; alert logged
TC4 Daily limit	Rs. 20,000 limit, request Rs. 20,100	Transaction rejected
TC5 Cash-out / power failure	Failure mid-transaction	Transaction reversed; account not debited

5. How Process Consistency Reduces Defects

When every release follows the same process, test cases are reused, regression suites grow, and past mistakes are not repeated. Reviews at each phase catch defects where they are introduced, so fewer escape to later phases. Standard templates and exit criteria remove personal variation among testers, making results predictable and comparable across releases. Defect trends can then be analysed and prevention actions taken.

6. Expected Outcomes

- Consistent and repeatable ATM behaviour across releases.
- Higher defect detection in early phases and a lower cost of fixing.
- Fewer field failures such as wrong debits, cash mismatch and card retention errors.
- Measurable quality through coverage and defect metrics, and improved customer confidence.

Summary of Key Points

- Every SDLC phase has a matching test activity (V-model), so defects are caught where they are introduced.
- A defined six-stage test process with entry/exit criteria makes ATM testing repeatable.
- Withdrawal tests must cover valid, insufficient funds, wrong PIN, limit and failure/recovery cases.
- Consistency lowers defects, because past defects are not repeated and regression tests grow with each release.

Answer 3

1. Introduction

Whenever the actual output of a web application differs from the expected output, testers must be able to describe exactly what has gone wrong. The basic testing vocabulary (as given by IEEE and ISTQB) separates the human mistake, its manifestation in the code, and the visible effect. Using the terms correctly makes analysis, tracking and communication accurate.

2. Basic Definitions

- **Error (mistake):** A human action that produces an incorrect result, such as a developer misunderstanding that the login must be case-sensitive.
- **Defect (fault, bug):** The incorrect step, condition or statement in the code or document that results from an error, e.g. a comparison written with equals-ignore-case for passwords.
- **Failure:** The deviation of the system from expected behaviour when the defect is executed, e.g. a user logs in with the wrong-case password.
- **Test case:** Inputs, preconditions and expected results used to check a specific condition.

3. Relationship Between Error, Defect and Failure

An error made by a person introduces a defect into the work product; when the defective code is executed under certain conditions it may cause a failure. The chain is therefore Error, then Defect, then Failure. Not every defect leads to a failure, because the defective path may never run, and some failures are caused by the environment rather than by code.

Scenario	Error	Defect	Failure
Login accepts any password after a valid user ID	Developer misread the authentication rule	Password check skipped when the user ID exists	Unauthorised user gains access
Transfer of Rs. 1,000 debits Rs. 1,100	Wrong understanding of fee rule	Fee added twice in the calculation	Incorrect balance displayed
Session does not expire after logout	Requirement for session timeout ignored	Session token is not invalidated	Back button shows the previous account page

4. Effect of Improper Understanding of These Terms

- **Wrong analysis:** If a failure is treated as the defect, testers may report symptoms and developers may fix the symptom, not the root cause.
- **Poor metrics:** Counting failures as defects distorts defect density and quality reports.
- **Communication gaps:** Reports such as "login is broken" give no steps or evidence, causing rework, rejection and delays.

- **Missed prevention:** Without identifying the error (origin), process weaknesses such as unclear requirements are never corrected.

5. Improving Defect Reporting Clarity

A good report describes the failure objectively and gives the developer all information needed to reproduce it. Recommended fields are listed below.

- Unique ID and a short, specific title (e.g. "Login succeeds with incorrect password in upper case").
- Environment (browser, version, OS, build number) and preconditions.
- Numbered steps to reproduce, test data used, expected result and actual result.
- Severity, priority, screenshots or logs, and the suspected module.
- Status tracking (New, Assigned, Fixed, Retested, Closed) in a tool such as Jira, with one defect per report.

6. Justification and Conclusion

In the given web application, the mismatch observed during login and transaction is a failure; the faulty password comparison and duplicated fee logic are defects; and the misread business rules are errors. Using these terms precisely allows the team to trace the failure to its source, fix the cause, and prevent recurrence. Clear, structured reports further reduce rework and improve overall software quality.

Summary of Key Points

- Error is the human mistake, defect is the fault in the work product, failure is the visible deviation in behaviour.
- Mixing the terms leads to wrong analysis, poor metrics and ineffective fixes.
- A good defect report has a specific title, environment, steps, expected and actual results, severity, priority and evidence.

Answer 4

1. Introduction

Software testing principles are general guidelines that help testers plan and execute testing effectively. The observation that a small number of modules contain most of the defects is an example of defect clustering, closely related to the Pareto principle: roughly 80% of defects are found in about 20% of modules. Applying the principles lets the team focus effort where it matters most and recover the schedule.

2. Testing Principles and Their Use in Planning

- **Testing shows the presence of defects:** Testing reduces the probability of hidden defects but cannot prove correctness, so exit criteria must be risk-based.
- **Exhaustive testing is impossible:** All input combinations cannot be tested; therefore techniques and priorities are used.
- **Early testing:** Reviews and tests begin in the requirement and design phases, since early defects are cheaper to fix.
- **Defect clustering:** Defects tend to cluster in complex, frequently changed or poorly understood modules; test effort is allocated accordingly.
- **Pesticide paradox:** Repeating the same tests stops finding new defects; test cases must be reviewed and updated regularly.
- **Testing is context dependent:** A payment module needs different testing from a content website.
- **Absence-of-errors fallacy:** A defect-free system is still a failure if it does not meet user needs.

3. Addressing the Defect Concentration Problem

- Collect defect data per module from the defect repository and rank modules by defect density.
- Analyse root causes of the clustered modules: complexity, requirement volatility, new technology, or inexperienced developers.
- Increase review and testing depth in these modules (code inspection, additional white-box tests, more regression).
- Refactor or redesign the worst modules and add developer unit tests before they come to the testers.
- Continue sampling testing in other modules so that new clusters are not missed.

4. Prioritising Test Cases

Test cases are ranked by combining risk (likelihood x impact), business criticality, defect history, requirement priority and frequency of use. High-priority cases run first, so the most serious problems are exposed early.

Priority	Selection Criteria	Example
P1 (Critical)	High risk, high usage, defect-prone modules	Payment authorisation, login
P2 (High)	Core features with moderate defect history	Order placement, cart update
P3 (Medium)	Less used features	Profile editing, wish list
P4 (Low)	Cosmetic and rarely used functions	Help page layout

5. Real-time Example

In an online shopping project, defect logs of three builds showed that the Payment and Order modules held 70% of all defects. The test manager shifted 60% of the effort to these modules, added boundary and decision-table tests, and introduced daily regression runs. After two builds, defect leakage fell sharply, and the team met the release date because the remaining modules, with few defects, required less testing.

6. Improvement in Testing Efficiency

Focusing on clusters raises the defects found per hour of testing, reduces redundant tests, lowers the cost of fixing, and makes schedules predictable. Combined with early testing and regularly refreshed tests (against the pesticide paradox), the team achieves higher quality with the same resources.

Summary of Key Points

- Defect clustering (Pareto 80/20) tells us where to concentrate testing.
- Other principles guide planning: early testing, no exhaustive testing, pesticide paradox and context dependence.
- Prioritise test cases by risk, usage and defect history so serious problems appear early.
- Result: higher defects found per testing hour and a recovered schedule.

Answer 5

1. Introduction

A defect repository is a central database that stores all defects with their attributes (ID, description, module, type, severity, origin, status, root cause and fix). Used only as a "bug list", it adds little value. Used as a source of data for classification, analysis and prevention, it breaks the cycle of repeated defects across releases.

2. How Defects Originate

- **Requirements phase:** Ambiguous, incomplete or conflicting requirements.
- **Design phase:** Incorrect algorithms, poor interface or database design.
- **Coding phase:** Logic mistakes, wrong data types, missing validations, misuse of APIs.
- **Testing phase:** Wrong test data or wrongly configured environments (defects in test ware).
- **Maintenance phase:** Changes that introduce side-effects (regression defects).

Behind each origin lie human causes such as poor communication, lack of skill, time pressure and inadequate reviews.

3. Defect Classification

Class	Description	Example
Requirement / specification	Missing, wrong or unclear requirement	Interest rate rule not specified
Design	Incorrect design logic or interface	Wrong module interface for statement generation
Coding	Control/logic, data, algorithmic or syntax errors	Loop runs one extra time
Interface / integration	Mismatch between modules or systems	Date format differs between modules
Testing / documentation	Wrong test cases, outdated manuals	Expected result recorded incorrectly

Defects are also classified by severity (critical, major, minor, trivial), priority, module, phase of detection and root cause, which allows multi-dimensional analysis.

4. Using Repository Data for Test Design and Prevention

- **Test design:** Defect-prone modules and defect types guide error-guessing and the selection of boundary, decision-table or state-based tests; every fixed defect becomes a regression test case.
- **Pattern identification:** Pareto charts and trend reports show the modules and defect types that recur.

- **Prevention:** Root-cause analysis (fishbone, 5-Whys) turns patterns into preventive actions such as checklists, coding standards and requirement-review templates.
- **Planning:** Defect density and detection-percentage metrics support effort estimation and release decisions.

5. Improvements to Avoid Recurring Defects

- Define a mandatory, standard defect template with classification fields, and train all team members to use it.
- Check the repository for duplicates and known issues before logging and before each release.
- Run a monthly defect-review meeting and perform causal analysis on high-severity and recurring defects.
- Link defects with requirements, test cases and code changes for traceability.
- Add every escaped defect to the automated regression suite and update coding checklists.

6. Conclusion and Expected Improvements

When the repository is used actively, recurring defects drop, test cases become more targeted, defect detection in earlier phases improves, and rework cost falls. The company can expect more stable releases, better predictability and increased customer satisfaction.

Summary of Key Points

- Defects originate in requirements, design, coding, testing and maintenance, driven by human and process causes.
- Classify by type, severity, origin phase, module and root cause so that patterns can be analysed.
- Use the repository for test design, regression suites and prevention actions such as checklists and reviews.

Answer 6

1. Introduction

Frequent crashes during a fund transfer indicate defects that exist in the application, network handling, back-end integration or device-specific behaviour. Applying the testing principles (early testing, defect clustering, context dependence and exhaustive testing being impossible) allows a methodical investigation instead of random trials.

2. Identifying the Root Cause

- **Reproduce the failure:** Record the exact steps, device, OS version, network type, app build and data that cause the crash.
- **Collect evidence:** Analyse crash logs (Logcat, Xcode Organizer, Firebase Crashlytics), server logs and API responses.
- **Isolate the layer:** Test the UI, the API (Postman) and the database independently to see where the failure begins.
- **Vary one factor at a time:** Change device, OS, network, amount or beneficiary type to find the condition that triggers the crash.
- **Perform root-cause analysis:** Use 5-Whys or a fishbone diagram to move from the symptom to the underlying defect.

Typical causes found are unhandled null values for the beneficiary, memory leaks, timeouts on slow networks, double-tap submission causing duplicate requests, incompatibility with specific OS versions, and unhandled exceptions when the server returns an error.

3. How Systematic Testing Reduces Such Issues

- **Planned coverage:** Test cases derived from requirements and risk cover valid, invalid, boundary and exceptional flows of the transfer.
- **Technique-based design:** Boundary value analysis for amounts and limits; equivalence classes for beneficiary types (own bank, other bank, IMPS/NEFT/UPI); state-transition tests for transaction status (initiated, pending, success, failed).
- **Non-functional testing:** Performance, stress, interruption (calls, low battery, app switching), network switching and security tests.
- **Regression:** Automated tests ensure that the fix for one crash does not cause another.

4. Illustrative Examples

Test Scenario	Condition	Expected Behaviour
---------------	-----------	--------------------

Network loss during transfer	Switch to airplane mode after pressing Submit	App shows status message; no crash; no duplicate debit
Double tap on Submit	Two rapid taps	Only one transaction is created
Maximum amount	Enter limit and limit plus one	Limit accepted; limit plus one rejected with message
Low-memory device	Run on a 2 GB RAM device with many background apps	Transfer completes without crash
Incoming call	Call arrives at the OTP screen	App resumes with the session intact

5. Improvements in Testing Practices

- Test on a device matrix of popular devices, OS versions and screen sizes using real devices and cloud farms.
- Introduce API, contract and automated UI testing (Appium, Espresso, XCUITest) in the CI pipeline.
- Perform monkey/exploratory testing and load tests with realistic concurrent users.
- Add crash monitoring in production with alerts, and feed each crash into the defect repository and regression suite.
- Include testers in design and code reviews to catch missing exception handling early.

6. Conclusion

By reproducing, isolating and analysing crashes methodically, and by strengthening test design, device coverage and automation, the team will reduce crashes in fund transfers, protect customers from financial inconsistencies and raise the reliability of the mobile banking application.

Summary of Key Points

- Reproduce, collect logs, isolate the layer and vary one factor at a time to find the root cause.
- Test amount boundaries, network loss, double submission, low memory, interruptions and OS/device variety.
- Add automation, device-matrix testing, performance tests and production crash monitoring to prevent recurrence.

Answer 7

1. Introduction

Testing and debugging are related but different activities. Testing is the process of executing software (and reviewing work products) to detect failures and evaluate quality. Debugging is the process of locating, analysing and removing the cause of a failure that testing has revealed. Confusing them causes testers to fix code, developers to skip proper testing, and defects to be handled ineffectively.

2. Differences Between Testing and Debugging

Basis	Testing	Debugging
Purpose	Find failures and assess quality	Find the cause of the failure and fix it
Performed by	Testers (also developers for unit tests)	Developers
Input	Test cases, requirements	Defect report, logs, code
Output	Test results, defect reports, metrics	Corrected code, fix notes
Nature	Planned, can be automated and scheduled	Reactive; depends on the defect
Knowledge needed	Requirements, techniques, domain	Code structure, design, programming
Result	Shows presence of defects	Removes defects; retest confirms the fix

3. Roles in Software Quality

- **Testing:** Gives visibility of product quality, verifies conformance to requirements, supports risk-based decisions and prevents defects through early reviews.
- **Debugging:** Corrects defects found, improves code quality and understanding, and provides root-cause data that helps prevent similar defects.
- **Together:** Testing finds, debugging fixes, and re-testing and regression testing confirm the fix and ensure that nothing else is broken.

4. Effects of Improper Usage

- If developers "test" only while debugging, they check only the failure they know about, so many other defects remain undetected.
- If testers try to debug, they lose time for test design and execution, and independence is lost.
- If debugging is called testing, there are no planned test cases, so there is no measure of coverage or quality.
- Fixes made without retesting and regression checking often introduce new defects.

- Absence of a clear defect report leads to long, wasteful debugging sessions.

Example: a login page shows an error for valid users. Testing exposes the failure and records steps to reproduce. The developer debugs and finds that the "isActive" flag is read wrongly. If the team stops after the fix, the same flag misuse in the password-reset module remains undiscovered. Planned retesting and regression testing would reveal it.

5. Structured Approach to Integrate Both

- Test and log: execute planned test cases and record a complete defect report.
- Triage: assign severity and priority and allocate the defect to a developer.
- Debug: reproduce, locate the root cause using logs, breakpoints and tracing, and fix it; also search for similar defects elsewhere.
- Unit test and code review of the fix by the developer.
- Retest: the tester verifies the fix with the original test case (confirmation testing).
- Regression: run the related regression suite to confirm that the change has not affected other features.
- Close the defect and record the root cause for process improvement.

6. Conclusion

Testing and debugging are complementary: testing detects, debugging diagnoses and corrects. Maintaining the distinction and following a defined defect life cycle improves defect resolution quality and software reliability.

Summary of Key Points

- Testing detects failures; debugging finds and removes their cause.
- Both are needed; retesting and regression testing close the loop after every fix.
- Following the defect life cycle (log, triage, debug, fix, retest, regress, close) integrates them effectively.

Answer 8

1. Introduction

Cross-browser testing verifies that a web application behaves and looks consistently on different browsers, versions, operating systems and devices. Because exhaustive testing of every combination is impossible, the strategy must be risk-based and driven by usage data, in line with the principles of early testing, defect clustering and context dependence.

2. How Defects Originate in Cross-Browser Scenarios

- **Different rendering engines:** Blink (Chrome, Edge), Gecko (Firefox) and WebKit (Safari) interpret HTML, CSS and JavaScript slightly differently.
- **Non-standard or unsupported features:** Use of newer CSS/JS features unavailable in older versions.
- **Vendor prefixes and defaults:** Different default styles, form controls and font rendering.
- **Poor development practices:** Browser sniffing, hard-coded pixel layouts and missing polyfills.
- **Environment factors:** Screen resolution, plug-ins, cookie and privacy settings, and OS differences.
- **Requirement gap:** The supported browsers and versions are not clearly defined.

3. Cross-Browser Test Strategy

- **Define scope:** Use analytics to pick the top browsers, versions and devices (e.g. Chrome, Safari, Firefox, Edge on Windows, macOS, Android, iOS) and document the compatibility matrix.
- **Prioritise:** Test critical flows first: search, product page, cart, checkout and payment, login.
- **Design test cases:** Functional, UI/layout, responsive (breakpoints), performance and accessibility checks per browser.
- **Execute:** Run in parallel on browser clouds; compare screenshots to a baseline.
- **Report and retest:** Log defects with browser and version, retest and add to regression.
- **Monitor:** Refresh the matrix every release as browser usage changes.

4. Example Test Matrix

Feature	Chrome	Firefox	Safari	Edge
Product image zoom	Pass	Pass	Fail - zoom not triggered	Pass
Date picker on checkout	Pass	Fail - format differs	Pass	Pass

Payment form validation	Pass	Pass	Pass	Pass
Responsive menu (mobile)	Pass	Pass	Fail - overlap	Pass

The failures in the matrix are logged against the specific browser, traced to unsupported CSS and JavaScript date handling, fixed with standard-compliant code or polyfills, and added to the regression suite.

5. How Systematic Testing Improves Reliability

A documented matrix, prioritised flows and automation give repeatable and measurable coverage. Compatibility defects are found before customers meet them, so cart abandonment and support calls decrease. Automated visual and functional regression ensures that every new release keeps behaviour consistent.

6. Tools and Techniques for Better Coverage

- **Tools:** Selenium WebDriver, Playwright, Cypress, BrowserStack, LambdaTest, Sauce Labs, and Percy or AppliTools for visual comparison.
- **Static checks:** HTML/CSS validators, ESLint, and caniuse.com for feature support.
- **Techniques:** Equivalence partitioning of browsers and versions, pairwise (combinatorial) testing of browser-OS-device combinations, responsive design testing, exploratory testing and CI-integrated nightly runs.

Summary of Key Points

- Cross-browser defects arise mainly from rendering-engine differences, unsupported features and poor coding practice.
- Use analytics to build a compatibility matrix, then test critical flows first.
- Automation, cloud browser farms, visual comparison and pairwise testing give wide coverage with manageable effort.

Answer 9

1. Introduction

Incorrect salary calculations caused by logical errors are serious because they affect employees directly, and may cause legal and financial consequences. Defect classification groups defects by their nature and origin so that testers know where to look and what to test.

2. Categorising the Payroll Defects

Defect Class	Meaning	Payroll Example
Control / logic	Wrong conditions or flow of control	Overtime paid when hours greater than or equal to 40 instead of greater than 40
Computational / algorithmic	Wrong formula or order of operations	Tax calculated on gross instead of taxable income
Data	Wrong data type, initialisation or boundary	Integer division truncates allowance; variable not reset between employees
Interface	Mismatch between modules	Attendance module sends days, payroll expects hours
Requirement	Misinterpreted business rules	Tax slab update not captured
Documentation / configuration	Outdated rules or tables	Old PF rate in configuration table

The given problem mainly belongs to the control (logic) and computational classes, with data defects as a secondary category. Defects are further tagged by severity (high, since salary is wrong), by detection phase and by root cause.

3. How Understanding Defect Classes Improves Testing

- **Targeted test design:** Logic defects call for decision-table, condition and branch coverage; computational defects call for boundary values and formula checks against independently calculated results.
- **Better error guessing:** Known classes (off-by-one, rounding, wrong operator) become a checklist of checks.
- **Effective review:** Reviewers look for typical defect types in code and requirements.
- **Prioritisation and metrics:** The class with the highest defect count receives extra attention.

4. Example Test Design

Rule: overtime is paid at 1.5 x hourly rate for hours above 40. Boundary tests: 39, 40, 41 hours. Expected pay for 41 hours at Rs. 200/hour is $40 \times 200 + 1 \times 300 = \text{Rs. } 8,300$. A decision table covers combinations such as full-time or part-time, with or without bonus, and tax slab. Results are compared with an independent spreadsheet (test oracle).

5. Prevention in Future Releases

- Review requirements with business rules and sign off examples with the finance team.
- Use code reviews and static analysis with a checklist for conditional operators and formulas.
- Unit test every calculation method with boundary values, and keep parameters (tax slabs, rates) in configurable tables with versioning.
- Record each defect with its class and root cause, and add it to the automated regression pack.

6. Improvements in Testing Strategy

- Adopt risk-based testing with the highest priority for salary, tax and deduction modules.
- Perform parallel testing: run the old and new systems on the same data and compare outputs.
- Use data-driven automation with large sets of employee scenarios (joiners, leavers, leave without pay, arrears).
- Include end-to-end payroll cycle tests and a user acceptance test with payroll staff before each release.

Summary of Key Points

- The payroll defects fall mainly in the control (logic) and computational classes, with data defects as a secondary class.
- Class-based thinking gives targeted tests: BVA, decision tables and independent calculation of expected pay.
- Reviews, unit tests, configurable rate tables and regression packs prevent recurrence; parallel testing adds confidence.

Answer 10

1. Introduction

Executing random tests without design leaves coverage unknown, wastes effort and misses defects. Viewing testing as an engineering activity means that it follows defined steps: analysis of the test basis, design of test cases, implementation, execution, evaluation and improvement, all supported by documents and measurements.

2. Redesigned Test Process

- **Analyse the test basis:** Study requirements, user stories and design to identify test conditions.
- **Plan:** Decide scope, techniques, priorities, tools, effort and exit criteria.
- **Design test cases:** Write cases with ID, objective, preconditions, input data, steps and expected results.
- **Review:** Peers and business analysts review the cases for correctness and coverage.
- **Execute and log:** Run, record actual results and report defects.
- **Measure and improve:** Track coverage, defect detection percentage and update the cases.

3. Importance of Structured Test Case Design

- **Coverage:** Every requirement is traced to test cases, so gaps are visible.
- **Efficiency:** Techniques give a small set of tests with high defect-finding potential, avoiding redundant tests.
- **Repeatability:** Documented cases can be re-run by any tester and automated for regression.
- **Early defect detection:** Designing tests exposes ambiguities in requirements before coding.
- **Measurability and accountability:** Pass/fail status, coverage and effort are quantifiable.
- **Knowledge retention:** Cases remain useful when team members change.

4. How Structured Design Improves Defect Detection

Random Testing	Structured Testing
Tests whatever comes to mind	Tests derived systematically from requirements
Edge conditions often missed	Boundary and negative conditions deliberately included
Same paths tested repeatedly	Redundant cases removed
No evidence of coverage	Coverage measured and reported
Defects hard to reproduce	Steps and data recorded for reproduction

Example: a field accepts age from 18 to 60. A random tester enters 25 and 30 and sees no failure. Boundary value analysis tests 17, 18, 19, 59, 60 and 61, and finds that the system accepts 61

because the condition is written as "age less than or equal to 61". Equivalence partitioning adds an invalid class of non-numeric and blank values.

5. Techniques to Enhance Test Effectiveness

- **Black-box:** Equivalence partitioning, boundary value analysis, decision tables, state-transition testing, use-case testing and cause-effect graphing.
- **White-box:** Statement, branch, condition and path coverage with unit-testing tools.
- **Experience-based:** Error guessing, checklists and exploratory testing with time-boxed charters, used to complement design techniques.
- **Support:** Pairwise testing for many parameters, risk-based prioritisation, automation of regression, and reviews of test cases.

6. Conclusion

Replacing random testing with a structured process improves coverage, repeatability and defect detection. The team gains confidence in release quality and reduces rework and cost.

Summary of Key Points

- Structured test case design gives coverage, repeatability and early detection of requirement gaps.
- Techniques such as BVA and equivalence partitioning find defects that random tests miss.
- Combine design techniques with experience-based and exploratory testing for best effectiveness.

Answer 11

1. Introduction

In a healthcare system, patient registration, appointments, billing, pharmacy and laboratory modules share the same patient data. Inconsistency, such as a different allergy list or patient age in two modules, can endanger patient safety and violate privacy regulations. Testing principles such as early testing, defect clustering, and context dependence (safety-critical software needs rigorous testing) guide the analysis.

2. Analysing the Issue

- Collect the inconsistent cases and identify which modules, fields and operations are involved.
- Trace the data flow: where the data is entered, transformed, stored and displayed.
- Compare data in the database tables, API responses and screens to locate the first point of divergence.
- Check the defect history to see whether the same modules or fields were affected earlier (clustering).
- Apply root-cause analysis to determine whether the cause lies in requirements, design, code or configuration.

3. How Defects Originate During Development

- **Requirements:** Data definitions, formats and ownership of fields not clearly specified for each module.
- **Design:** Duplicate data storage, missing foreign keys, no single source of truth, wrong transaction boundaries.
- **Coding:** Inconsistent validation, different data types or date formats, not updating all linked tables, race conditions.
- **Integration:** Interface mapping errors, character-encoding problems, failed or partial message delivery between modules (e.g. HL7/FHIR messages).
- **Change management:** A modification in one module without impact analysis on others.

4. Systematic Testing for Data Integrity

Test Type	What is Checked	Example
Field validation	Format, range and mandatory checks	DOB cannot be a future date; age computed consistently
Integration testing	Data passing between modules	Patient demographic update in registration appears in billing

Database testing	Referential integrity, constraints, ACID properties	No billing record without a valid patient ID
End-to-end scenario	Complete patient journey	Admit, prescribe, dispense, bill, discharge
Concurrency testing	Simultaneous updates	Two users editing one record
Regression testing	Existing data after changes	Old records after a database upgrade

Example: a patient's allergy to penicillin is entered in the clinical module. Test cases verify that the pharmacy module shows the same allergy and blocks the prescription. A failure here would reveal the missing synchronisation defect.

5. Improvements in Defect Tracking and Reporting

- Use a defect tool (Jira, Bugzilla) with mandatory fields: module, data affected, severity (patient safety impact), steps and evidence with masked patient data.
- Classify defects as data, interface, logic or requirement and link them to the requirements and test cases.
- Maintain a defect life cycle with ownership and time limits, and escalate patient-safety defects immediately.
- Review defect trends weekly, and add each fixed defect to the regression suite.
- Follow data-privacy norms (HIPAA, local rules) in test data and reports.

6. Conclusion

By tracing data flows, targeting high-risk modules, testing integration and database integrity, and managing defects through a disciplined process, the team can eliminate inconsistency and ensure reliable healthcare data.

Summary of Key Points

- Data inconsistency usually originates in unclear data definitions, duplicate storage, interface mapping errors and uncontrolled changes.
- Integration, database, end-to-end and concurrency tests protect data integrity.
- Patient-safety defects need immediate escalation and masked test data in reports.

Answer 12

1. Introduction

Repeated defects show that the lessons from earlier releases are not being captured or used. A defect repository is the organisational memory of all defects found in testing and in production. Analysing it systematically reveals why the defects recur and what must change in the testing and development process.

2. Analysing the Issue Using the Repository

- Extract defects of the last several releases and group them by module, type, severity, origin phase and root cause.
- Identify duplicates and re-opened defects to see whether fixes were incomplete.
- Check whether each fixed defect has a corresponding regression test.
- Compare the phase where defects were introduced with the phase where they were found (defect leakage).
- Review the root-cause field; if it is blank or generic, the process itself needs improvement.

3. How Repository Data Supports Test Design

- **Risk-based selection:** Modules with high defect density get more and deeper tests.
- **Error guessing:** Typical defect types (boundary errors, null handling, date formats) become targeted test ideas and checklists.
- **Regression suites:** Every defect becomes a test case, preventing reappearance.
- **Test improvement:** Defects that escaped show weaknesses in current test cases, which are then revised (reducing the pesticide paradox).
- **Effort and schedule:** Historical data supports estimates and release readiness decisions.

4. Identifying Defect Patterns

Patterns are identified by simple analytical tools: Pareto charts (a few modules or causes account for most defects), trend charts (defects per release or build), defect-density by module, cross-tabulation of type against module, and fishbone diagrams for root causes.

Release	Total Defects	Validation Defects	Calculation Defects	Re-opened
R1	60	22	15	6
R2	55	24	14	9
R3	58	26	16	11

The table shows that input-validation defects and re-opened defects increase from release to release. The pattern reveals that validation rules are not covered in unit or regression tests and that fixes are not verified properly, so these two areas become the targets for corrective action.

5. Improvements to Prevent Recurring Defects

- Mandatory root-cause entry and classification when a defect is closed.
- Add validation checklists to requirement and code reviews, and static-analysis rules for the common patterns.
- Automate regression tests for every fixed defect and run them in the CI pipeline.
- Hold defect-prevention (causal analysis) meetings after each release and track actions.
- Search the repository before logging, to reuse knowledge and avoid duplicates; train the team on the use of the data.
- Define metrics such as defect leakage and re-open rate with targets, and review them regularly.

6. Conclusion

An actively used defect repository turns past defects into preventive knowledge. The company will see fewer repeated defects, more effective test cases and improved release stability.

Summary of Key Points

- The repository is the organisation's memory; analyse it by module, type, root cause and phase.
- Every fixed defect should become a regression test and every pattern a preventive action.
- Track metrics such as defect leakage and re-open rate to confirm improvement.

Answer 13

1. Introduction

A learning system is used heavily at particular times, such as exam registration, assignment deadlines and live classes. Failure at these peaks shows that load-related scenarios were not considered during test planning. The principles "exhaustive testing is impossible" and "testing is context dependent" imply that tests must be chosen according to the actual usage profile and risks.

2. Gaps in Testing

- No performance, load, stress or endurance tests were planned; only functional tests were run.
- No operational profile: expected number of concurrent users, peak hours and transaction mix were not defined.
- Test environment was smaller than production, or test data was too small.
- Non-functional requirements (response time, throughput, availability) were not specified or measured.
- Third-party services (video streaming, payment, authentication) were not tested under load.
- No monitoring or capacity planning, and no tests of recovery or failover.

3. Impact of Test Planning and Coverage on Performance

Test planning decides what is to be tested, with which data, tools and environment. If non-functional objectives are missing from the plan, performance is never evaluated. Coverage must include not only functional requirements but also load levels, user types, data volume and infrastructure conditions. Good coverage reveals bottlenecks (database queries, memory, thread pools, network) before users do.

Test Type	Objective	Example for the Learning System
Load testing	Behaviour under expected peak	5,000 students logging in and opening a quiz
Stress testing	Breaking point and recovery	10,000 users until failure; check graceful degradation
Spike testing	Sudden surge	Thousands submit assignments at deadline
Endurance (soak)	Stability over time	Live classes for 8 hours; check memory leaks
Scalability testing	Effect of adding resources	Add servers and measure response time

4. Better Test Strategies and Reliability

- Define service-level targets (e.g. 95% of pages in under 3 seconds, error rate under 1%).
- Build realistic workload models from production analytics and run tests in a production-like environment.
- Test early and repeatedly: performance checks in each sprint rather than only before release.
- Analyse results to find and fix bottlenecks, then retest to confirm improvement.

Example: a load test of 3,000 concurrent users shows that quiz submission response time jumps from 2 to 14 seconds because of a missing database index. Fixing it and retesting brings the response time below 3 seconds. Without the test, the failure would have occurred during the live examination.

5. Enhancements to Avoid Such Failures

- Include performance and security testing in the master test plan with tools such as JMeter, Gatling, k6 or LoadRunner.
- Introduce monitoring (APM tools, dashboards) and alerts, and use the findings for capacity planning and auto-scaling.
- Use caching, CDN for video and database optimisation based on test results.
- Perform failover and recovery tests, and a pre-peak readiness test before each academic cycle.
- Review production incidents and add them to the test suite.

Summary of Key Points

- Peak-time failure is mostly a non-functional testing gap: load, stress, spike and endurance tests were missing.
- Test plans must define performance targets, realistic workload models and production-like environments.
- Monitoring, capacity planning and pre-peak readiness tests keep the system reliable.

Answer 14

1. Introduction

Testers are often brought in only after coding, when defects are expensive to correct. The principle of early testing says that testing activities should start as early as possible in the life cycle. The role of the tester therefore extends from requirement analysis to maintenance, not only to test execution.

2. Role of the Tester Across SDLC

SDLC Phase	Tester's Role
Requirements	Review for ambiguity, completeness and testability; define acceptance criteria
Design	Review design, identify risks, prepare test strategy and test plan
Coding	Design test cases and data, support unit testing, review code with developers, prepare automation
Testing	Execute tests, report defects, perform confirmation and regression testing
Deployment	Smoke and acceptance tests, release readiness report
Maintenance	Regression tests for changes, analysis of production defects

3. Impact of Early Involvement on Quality

- **Defect prevention:** Defects in requirements and design are found before they are coded; the cost of correction rises rapidly in later phases.
- **Better requirements:** Testers ask "how will we verify this?", which exposes vague statements.
- **Better test coverage:** Test design starts early, so tests are ready when the build arrives and the test phase is shorter.
- **Shared quality ownership:** Developers write more testable code when they know the test conditions.
- **Reduced rework and schedule risk:** Fewer late surprises and fewer emergency fixes.

4. Practical Example

In an online ticket-booking project, a tester reviewing the requirement "the system shall allow a user to cancel a ticket" asks about the refund rules, time limits, and partial cancellations. These were not defined. The business analyst updates the requirement before design. Had the tester joined only at system testing, the missing rules would have been coded incorrectly, causing financial defects found late and requiring redesign of the cancellation module. The early review cost a few hours, while the late fix would have taken days.

5. Changes in Team Collaboration Practices

- Include testers in requirement, design and sprint-planning meetings, and in the definition of "ready" and "done".
- Adopt the three-amigos session (business, developer, tester) for every user story, with acceptance criteria written in a Given-When-Then format.
- Introduce peer reviews and walkthroughs of requirements, design and code with tester participation.
- Use shared tools (Jira, Confluence) and common dashboards so that requirements, tests and defects are linked.
- Encourage test-driven or behaviour-driven development, pair testing with developers, and continuous integration with automated tests.
- Hold joint retrospectives and share defect data, so quality becomes a team responsibility.

6. Conclusion

Involving testers from the start moves defect detection to the earlier phases, reduces cost and delay, and improves the overall quality of the product through better collaboration.

Summary of Key Points

- The tester contributes in every SDLC phase, from requirement review to maintenance.
- Early involvement prevents defects, improves requirements and shortens the test phase.
- Three-amigos sessions, shared tools and joint reviews make quality a team responsibility.

Answer 15

1. Introduction

Test coverage measures how much of the requirements, code or input space has been exercised by tests. When coverage is incomplete, the system works only for the scenarios that were tried. Because exhaustive testing is impossible, testers must select test cases so that the most important and most error-prone conditions are covered with a reasonable number of tests.

2. Identifying Coverage Gaps

- **Requirement coverage:** Use the traceability matrix to find requirements without test cases.
- **Code coverage:** Use coverage tools (JaCoCo, Cobertura, Istanbul) to find statements, branches and conditions never executed.
- **Input coverage:** Check which equivalence classes, boundaries and invalid inputs were not used.
- **Scenario coverage:** Look for missing exceptional paths, alternate flows, state changes and data combinations.
- **Defect analysis:** Failures reported from the field show the cases that tests missed.

3. How Proper Coverage Ensures Reliability

High and well-targeted coverage increases the probability that defects are found before release, shows which parts of the system are untested (and therefore risky), and provides objective evidence for exit decisions. Reliability improves because failures in rarely executed paths and unusual inputs are removed. Coverage does not prove the absence of defects (testing shows the presence of defects), so it must be combined with good test design.

Coverage Type	Meaning	Example
Statement	Every statement executed at least once	Every line in discount calculation
Branch / decision	Each true and false outcome of a decision executed	Both outcomes of "if amount > 1000"
Condition / path	All condition combinations or independent paths	Combinations of member status and coupon
Requirement	Each requirement linked to a test	Every item in the RTM

4. Improving Test Case Design

Example: a function gives a discount of 10% if the purchase is above Rs. 1,000 and the customer is a member. Tests with only (Rs. 1,500, member) pass, but the system fails for (Rs. 1,000, member), because the condition uses "greater than or equal". A better design uses:

- Boundary value analysis: 999, 1,000, 1,001.
- Equivalence partitioning: member, non-member, invalid membership value.
- Decision table: amount above/not above 1,000 against member/non-member (four rules).
- Negative tests: negative amount, zero, non-numeric input and null values.

5. Techniques for Better Validation

- Black-box techniques: equivalence partitioning, boundary values, decision tables, state transitions, use-case testing.
- White-box techniques: statement, branch and condition coverage; mutation testing to check the quality of tests.
- Pairwise testing for combinations of parameters, error guessing and exploratory testing.
- Reviews of test cases against the RTM, and use of coverage tools in the CI pipeline with minimum thresholds.
- Regression testing of every fixed defect and periodic refreshing of tests.

6. Conclusion

Identifying gaps through requirement and code coverage analysis, and improving the design with systematic techniques, makes the system dependable for all important inputs, not just the commonly tried ones.

Summary of Key Points

- Find gaps with requirement coverage (RTM), code coverage and input-space analysis.
- Coverage supports reliability but does not prove the absence of defects, so test design quality matters.
- Use BVA, equivalence partitioning, decision tables, branch coverage and mutation testing for better validation.

Answer 16

1. Introduction

In a logistics application (order booking, route planning, tracking, delivery confirmation), a delayed or unclear defect report slows the fix and can disrupt deliveries. A defect report is the main communication tool between testers, developers and managers, so its quality directly affects the speed of defect resolution.

2. Problems with Poor Reporting

- Vague titles such as "tracking not working" with no steps or data.
- Missing environment details (build, browser/device, test data), so developers cannot reproduce the failure.
- Confusion between error, defect and failure, and between severity and priority.
- Duplicate reports, defects reported late (in e-mails or chats) and no ownership.
- No status tracking, so defects are lost or re-opened without clear reasons.

3. How Clear Definitions Help Tracking

When everybody uses the same definitions of defect, failure, severity (impact on the system) and priority (urgency of the fix), reports can be classified, sorted and measured consistently. Clear definitions allow correct counting of defects per module, correct prioritisation (e.g. a wrong delivery address is high severity), unambiguous status in the life cycle (New, Assigned, Open, Fixed, Retest, Closed, Rejected, Deferred), and meaningful metrics such as defect density and age of open defects.

4. Structured Reports Enhance Communication

Field	Content (Example)
Defect ID / Title	LG-142: Delivery status stays "In Transit" after delivery confirmed
Module / Build	Shipment tracking / Build 2.4.1
Environment	Test server; Chrome 124; Windows 11
Steps to reproduce	1. Log in as driver. 2. Open shipment S-1008. 3. Press "Confirm Delivery".
Expected result	Status changes to "Delivered"; customer receives SMS
Actual result	Status remains "In Transit"; no SMS sent
Severity / Priority	Major / High
Attachments	Screenshot, API log, test data
Reported by / Date, Status	Tester name / date; New

Such a report lets the developer reproduce and fix the defect without further questions. The project manager can quickly assess impact, and the tester can retest with the same steps.

5. Improvements in Reporting Standards

- Adopt a standard template following IEEE 829 / ISTQB with mandatory fields, and a single defect-tracking tool (Jira, Bugzilla, Azure DevOps).
- Report each defect immediately after confirmation and one defect per report; search for duplicates first.
- Define severity and priority scales and the defect life cycle, with agreed response times (SLA) for each level.
- Be objective and factual, with no blame and with evidence (screenshots, logs, videos).
- Hold daily defect triage meetings, use dashboards for open defects, ageing and trends, and review report quality in retrospectives.

6. Conclusion

Clear definitions, structured reports and standard tools shorten defect identification and resolution time, improve communication between teams and give management a reliable view of quality in the logistics application.

Summary of Key Points

- Poor reports cause delays; clear definitions make tracking and metrics consistent.
- A structured template with steps, environment, expected/actual results and evidence lets developers reproduce defects quickly.
- Standard tools, severity and priority scales, SLAs and daily triage keep defects moving.

Answer 17

1. Introduction

When different modules are tested by different people in different ways, results vary: some modules are well tested and others are not. A process-based approach defines a common, documented test process (with standards, templates, techniques and criteria) that every module follows, so that testing quality does not depend on individuals.

2. Process-based Testing Approach

- **Define the test process:** Planning, analysis and design, implementation, execution, reporting and closure, with entry and exit criteria for each stage.
- **Standardise artefacts:** Common templates for test plans, test cases, defect reports and summary reports (IEEE 829).
- **Standardise techniques:** Guidelines on when to use equivalence partitioning, boundary values, decision tables and state tests, plus minimum coverage targets.
- **Define roles and responsibilities:** Test lead, testers, reviewers and developers with clear accountability.
- **Adopt a maturity model:** Models such as TMMi help the organisation move from ad-hoc testing to a defined and optimised process.

3. How Standardised Processes Improve Quality

- **Predictable results:** The same activities and criteria are applied to every module and release.
- **Higher coverage:** Mandatory design techniques and traceability avoid gaps.
- **Reusability:** Templates, test data and automated suites can be shared across modules.
- **Efficient onboarding:** New testers follow a known process and produce comparable work.
- **Measurable quality:** Common metrics (coverage, defect density, execution rate) allow comparison and improvement.

4. Example

An HR system has Leave, Attendance and Payroll modules. Before standardisation, Leave had detailed test cases, Attendance had only quick checks, and Payroll had no regression tests. After introducing the process, every module must pass the same gates, as shown below.

Stage	Standard Requirement	Applied to Every Module
Planning	Test plan from the common template	Scope, risks, schedule

Design	Techniques plus RTM; peer review	Minimum 100% requirement coverage
Execution	Daily status; standard defect report	Pass rate and defect metrics
Exit	No open critical defects; coverage achieved	Sign-off by test lead
Closure	Lessons learned and regression suite update	Stored in repository

As a result the Attendance and Payroll modules showed higher defect detection during testing, and the production defects across modules became comparable.

5. Methods to Ensure Consistency

- Publish a test policy and test strategy applicable to all modules and teams.
- Use a test management tool with configured templates and workflows.
- Conduct reviews and audits (process compliance checks) and test-case peer reviews.
- Provide training and checklists; assign a test lead or a test centre of excellence.
- Track common metrics on a dashboard and run retrospectives to improve the process continuously.

Summary of Key Points

- A common process, templates and technique guidelines make testing quality independent of individuals.
- Quality gates at planning, design, execution, exit and closure give the same rigour for every module.
- Audits, training, tools and shared metrics keep the process consistent over time.

Answer 18

1. Introduction

Edge cases are inputs or conditions at the extreme limits of normal operation, such as minimum and maximum values, empty inputs, very large data, or unusual combinations. Failures in production after deployment show that such cases were never considered in test design.

2. Why Edge Cases Were Missed

- Testing concentrated on typical "happy path" scenarios because of time pressure; exhaustive testing is impossible and no technique was used to select the important cases.
- Requirements did not state limits, formats or exceptions clearly, so testers did not know the boundaries.
- No boundary value analysis or negative test design; test data was simple and small.
- Test cases were repeated unchanged from release to release (pesticide paradox).
- Test environment or data did not match production (large volumes, special characters, time zones).
- Lack of reviews and of analysis of earlier defects.

3. How Boundary Testing Helps

Experience shows that many defects occur at the edges of input ranges because of mistakes in relational operators (less than versus less than or equal), off-by-one errors in loops and arrays, and wrong limits. Boundary value analysis (BVA) selects test values at and around each boundary, giving high defect-finding ability with few tests. Two-value BVA uses the boundary and the value just outside it; three-value BVA also uses the value just inside.

Requirement	Valid Range	BVA Test Values
Password length	8 to 16 characters	7, 8, 9, 15, 16, 17
Order quantity	1 to 100	0, 1, 2, 99, 100, 101
Withdrawal amount	Rs. 100 to Rs. 20,000	99, 100, 101, 19,999, 20,000, 20,001
Leap-year date entry	Valid dates	28 Feb, 29 Feb (leap), 29 Feb (non-leap), 31 Apr

4. Including Edge Cases in Testing

- Identify limits from requirements, database field definitions and business rules; clarify unclear limits with the analyst.
- Apply BVA and equivalence partitioning to every input; include empty, null, maximum length, special characters, Unicode, and negative and zero values.

- Add edge cases in time, volume and state: month-end, year-end, daylight-saving changes, maximum file size, zero records, millions of records, session timeout.
- Use error guessing and checklists from earlier defects and exploratory testing sessions.
- Review test cases for boundary coverage and run edge-case tests in an environment with production-like data.

Example: a coupon is valid for orders of Rs. 500 or more. The code uses "amount > 500".

Testing with Rs. 400 and Rs. 800 passes, but the boundary test with exactly Rs. 500 fails and exposes the defect before release.

5. Improvements in Test Design

- Make boundary and negative tests part of the test design standard and checklist.
- Strengthen requirement reviews to state ranges, formats and error messages.
- Use combinatorial and data-driven automation to run many edge values quickly.
- Update test suites regularly with escaped defects and monitor production incidents.
- Use code coverage and mutation testing to find untested conditions.

Summary of Key Points

- Edge cases are missed because of happy-path testing, vague requirements and repeated tests.
- Boundary value analysis targets the limits where off-by-one and operator errors occur.
- Include empty, maximum, negative, date/time and volume cases, with production-like data and regular test updates.

Answer 19

1. Introduction

Without proper documentation, defects are lost in e-mails, spreadsheets and conversations. Nobody knows which defects are open, who is working on them or why they occurred. A defect repository is a central, structured store of all defect information throughout the defect life cycle and is the basis of defect management.

2. Applying the Defect Repository Concept

- Select a single tool (Jira, Bugzilla, Mantis, Azure DevOps) as the only place where defects are recorded.
- Define a standard defect record: ID, title, description, module, build, environment, steps, expected and actual results, severity, priority, status, owner, dates, attachments, origin phase, defect type and root cause.
- Implement the defect life cycle: New, Assigned, Open, Fixed, Retest, Verified/Closed, with Reopened, Rejected and Deferred.
- Link defects to requirements, test cases, code changes and releases for traceability.
- Control access, back up the repository, and keep the history of changes.

3. How Documentation Improves Defect Management

- **Visibility:** All stakeholders see the current status and ownership of each defect.
- **Reproducibility:** Detailed steps and evidence reduce back-and-forth communication.
- **Prioritisation:** Severity and priority help to plan fixes according to business impact.
- **Accountability:** Every defect has an owner, a date and a history.
- **Knowledge retention:** Fix details help in solving similar problems and in training.
- **Audit and compliance:** Records provide evidence for customers and regulators.

4. How Structured Repositories Support Analysis

Analysis	Data Used	Insight Gained
Defect density	Defects per module / size	Identifies risky modules (clustering)
Defect trend	Defects opened and closed per week	Shows stability and release readiness
Ageing	Days open by severity	Reveals bottlenecks in fixing
Root-cause distribution	Origin phase and cause	Directs preventive actions (e.g. requirement reviews)
Detection effectiveness	Found in testing versus production	Measures defect leakage and test quality

Example: the repository shows that 40% of defects have the root cause "unclear requirements", and that the Billing module has the highest density. Management responds with stronger requirement reviews and extra testing in Billing.

5. Improvements in Documentation Practices

- Provide a mandatory template with dropdown values for severity, type and status, and examples of good reports.
- Train testers and developers, and make "no record, no fix" the rule.
- Update status in real time and add a fix description and root cause when a defect is closed.
- Perform periodic repository cleanup, a duplicate check and quality audits of the entries.
- Create dashboards and weekly reports, and review them in defect triage and retrospective meetings.

6. Conclusion

A structured defect repository with disciplined documentation makes tracking reliable, supports analysis and prevention, and improves communication and software quality.

Summary of Key Points

- One central repository with a standard record and life cycle removes lost and duplicate defects.
- Documentation gives visibility, reproducibility, accountability and audit evidence.
- Dashboards on density, ageing, trend and root cause support analysis and prevention.

Answer 20

1. Introduction

Transaction processing (deposits, withdrawals, transfers, bill payments) is the core of a banking system, and the accuracy, consistency and security of every transaction is critical. Repeated failures show that testing practices lack planning, depth and learning from past defects. The redesigned approach applies the seven testing principles within a structured process.

2. Redesigned Testing Approach

- **Early testing:** Review requirements and design for transaction rules; define acceptance criteria and begin test design in parallel with development.
- **Risk-based planning:** Rank features by financial and regulatory risk; fund transfer, settlement and reversal receive the highest priority.
- **Defect clustering and history:** Use the defect repository to concentrate tests in failure-prone modules.
- **Structured design:** Technique-based test cases traced to requirements, with reviews.
- **Test environment and data:** Production-like environment with masked data and simulators for external systems (payment gateway, core banking).
- **Execution, regression and reporting:** Automated regression suite, standard defect reports and metrics.
- **Avoid the pesticide paradox:** Review and update tests every release.

3. How Structured Testing Improves Quality

Planned coverage ensures that all transaction types and exceptional flows are tested, not only the common ones. Defects are found early, and fixes are verified by regression tests. Traceability and metrics provide evidence of quality for release decisions and audits. The result is fewer failed or duplicate transactions and higher customer trust.

4. Examples of Test Design

Scenario	Technique	Expected Outcome
Transfer limit Rs. 1 to Rs. 1,00,000	Boundary value analysis (0, 1, 1,00,000, 1,00,001)	Boundaries accepted or rejected correctly
Balance versus amount versus daily limit	Decision table	Correct approve/decline decision for each rule combination
Pending, success, failed, reversed status	State-transition testing	Only valid transitions allowed
Debit succeeds but credit fails	Failure and recovery test	Transaction rolled back; both accounts consistent

Two simultaneous withdrawals from one account	Concurrency test	No overdraft; correct final balance
Same request sent twice	Idempotency test	Only one debit

5. Techniques to Enhance Defect Detection

- Black-box: equivalence partitioning, boundary values, decision tables, state transitions, use-case and end-to-end tests.
- White-box: branch and condition coverage of calculation, fee and interest logic; static analysis and code reviews.
- Non-functional: performance and load tests (month-end, salary day), security tests (authentication, authorisation, injection) and recovery tests.
- Data and integration: reconciliation of ledger and account balances, ACID checks, API contract testing, and testing of interfaces with payment gateways.
- Experience-based: error guessing from past failures and exploratory testing.
- Automation and CI: nightly regression, with defect-trend dashboards and root-cause analysis for every escaped defect.

6. Conclusion

Applying the testing principles through a structured, risk-based and measurable process will reduce the repeated failures in transaction processing, improve the accuracy and security of the banking system and strengthen customer confidence.

Summary of Key Points

- Apply early testing, risk-based planning, defect clustering and regular test refresh to the banking system.
- Cover limits, rules, states, failure/recovery, concurrency and repeated requests in transaction tests.
- Combine black-box, white-box, non-functional, reconciliation and automated regression techniques for strong defect detection.